

Mục Lục

BÁO CÁO CHI TIẾT HỆ THỐNG QUẢN LÝ TỒN KHO BẰNG REINFORCEMENT LEARNING

2

1. PHÁT BIỂU BÀI TOÁN (PROBLEM FORMULATION)	2
2. THIẾT KẾ MDP (MARKOV DECISION PROCESS)	2
A. State Space (Không gian trạng thái - 2,646 trạng thái)	2
B. Action Space (Không gian hành động - 6 hành động)	3
C. Reward Function (Hàm phần thưởng)	4
D. Transition Dynamics (Động lực học môi trường)	4
E. Terminal và Truncated (Điều kiện kết thúc)	5
F. Giả định Markov (Markov Assumption Validity)	5
3. THUẬT TOÁN (ALGORITHMS IMPLEMENTATION)	6
A. Random Agent (Baseline)	6
B. Heuristic Agent (Baseline)	6
C. Q-Learning Agent (Off-policy TD Control)	6
D. SARSA Agent (On-policy TD Control)	6
E. Double Q-Learning Agent (Off-policy, giảm maximization bias)	6
F. Epsilon-greedy Exploration và Evaluation Mode	7
4. BỘ KIỂM THỬ MÔI TRƯỜNG (UNIT TESTS)	7
A. Kiểm thử Biên (Test Boundary) — [test_env.py](file:///d:/HocTap/HocMayNangCao/tests/test_env.py)	7
B. Kiểm thử Hành động không hợp lệ (Test Invalid Action) — [test_env.py](file:///d:/HocTap/HocMayNangCao/tests/test_env.py)	8
C. Kiểm thử Trạng thái kết thúc (Test Terminal State) — [test_env.py](file:///d:/HocTap/HocMayNangCao/tests/test_env.py)	8
D. Kiểm thử Hạt giống ngẫu nhiên (Test Seed) — [test_env.py](file:///d:/HocTap/HocMayNangCao/tests/test_env.py)	8
E. Kiểm thử Mã hóa/Giải mã (Test Encoder/Decoder) —	

[test_encoder.py](file:///d:/HocTap/HocMayNangCao/tests/test_encoder.py)

8

F. Kiểm thử Hàm Phần thưởng (Test Rewards) —

[test_rewards.py](file:///d:/HocTap/HocMayNangCao/tests/test_rewards.py)

8

5. THỰC NGHIỆM & PHÂN TÍCH (EVALUATION PIPELINE)	8
A. Quy trình thực nghiệm chuẩn hóa	9
B. Các chỉ số đo lường thống kê (Mean +- Std)	9
C. Phân tích kết quả thực nghiệm	9
6. CHI TIẾT CẤU TRÚC MÃ NGUỒN DỰ ÁN	10
7. HƯỚNG DẪN VẬN HÀNH VÀ CHẠY DỰ ÁN (WINDOWS)	11
Bước 1: Khởi tạo và cài đặt môi trường	11
Bước 2: Chạy kiểm thử tự động	11
Bước 3: Huấn luyện các Agent RL	11
Bước 4: Chạy Đánh giá	11
Bước 5: Tạo các biểu đồ báo cáo	12
Bước 6: Khởi chạy Web Dashboard	12

type: theory-note

tags: [note/theory, note/research]

title: Báo cáo Chi tiết Hệ thống Quản lý Tồn kho bằng RL

created: 2026-06-08

BÁO CÁO CHI TIẾT HỆ THỐNG QUẢN LÝ TỒN KHO BẰNG REINFORCEMENT LEARNING

Tài liệu này mô tả chi tiết kiến trúc, công thức, thuật toán, mã nguồn và kết quả thực nghiệm của dự án **Quản lý tồn kho bằng RL**, đối chiếu trực tiếp với các yêu cầu bắt buộc của môn học Học máy Nâng cao.

1. PHÁT BIỂU BÀI TOÁN (PROBLEM FORMULATION)

- Agent (Tác nhân):** Người quản lý kho hàng (Inventory Manager). Nhiệm vụ của Agent là quyết định số lượng hàng cần đặt mua thêm mỗi ngày.
- Environment (Môi trường):** Mô phỏng chuỗi cung ứng hàng hóa hàng ngày, bao gồm quá trình nhập hàng từ đơn đặt ngày hôm trước, phát sinh nhu cầu mua hàng ngẫu nhiên của khách hàng theo xu hướng thị trường, thực hiện bán lẻ, tính toán chi phí lưu kho và chi phí phạt thiếu hàng.
- Mục tiêu dài hạn:** Tối đa hóa tổng lợi nhuận tích lũy (Cumulative Profit) trong chu kỳ kinh doanh 30 ngày bằng cách:
 - Đáp ứng tối đa nhu cầu của khách hàng để tối đa hóa doanh thu.
 - Tránh tích trữ quá nhiều hàng trong kho để giảm thiểu chi phí lưu kho (Holding Cost).
 - Tránh để kho bị trống khi khách có nhu cầu mua để giảm thiểu chi phí phạt thiếu hàng (Stockout Penalty).
 - Tối ưu hóa chi phí đặt hàng (Purchase Cost).

2. THIẾT KẾ MDP (MARKOV DECISION PROCESS)

A. State Space (Không gian trạng thái - 2,646 trạng thái)

Trạng thái tại mỗi ngày được biểu diễn dưới dạng tuple 4 thành phần:

```
state = (inventory, demand_regime, day_of_week, pending_order)
```

1. **inventory** (Tồn kho): Số lượng hàng có sẵn trong kho vào đầu ngày sau khi đã nhận hàng đặt từ hôm trước. Lượng tồn kho tối đa là 20. Do đó, **inventory** nhận một trong các giá trị {0, 1, 2, ..., 20} (21 giá trị).

2. **demand_regime** (Xu hướng nhu cầu): Xu hướng tiêu dùng của thị trường, nhận một trong ba giá trị {0 (Low), 1 (Medium), 2 (High)} (3 giá trị).

3. **day_of_week** (Thứ trong tuần): Ngày hiện tại trong tuần, nhận một trong các giá trị {0 (Mon), 1 (Tue), 2 (Wed), 3 (Thu), 4 (Fri), 5 (Sat), 6 (Sun)} (7 giá trị).

4. **pending_order** (Hàng đang chờ giao): Số lượng hàng đã quyết định đặt mua từ hôm trước và sẽ về kho vào sáng mai, nhận một trong các giá trị {0, 1, 2, 3, 4, 5} (6 giá trị).

Tổng số lượng trạng thái lý thuyết:

```
21 (inventory) * 3 (demand_regime) * 7 (day_of_week) * 6 (pending_order) = 2,646 trạng thái
```

Cơ chế State Encoder & Decoder:

Để thuật toán dạng bảng (Tabular RL) có thể truy xuất bảng Q-table, tuple trạng thái được mã hóa thành một số nguyên index duy nhất **encoded_state** trong khoảng [0, 2645]:

- **Encoder (Mã hóa):**

```
encoded_state = (inventory * 126) + (demand_regime * 42) + (day_of_week * 6) + pending_order
```

- **Decoder (Giải mã):**

```
pending_order = encoded_state % 6
remainder1 = encoded_state // 6
day_of_week = remainder1 % 7
remainder2 = remainder1 // 7
demand_regime = remainder2 % 3
inventory = remainder2 // 3
```

B. Action Space (Không gian hành động - 6 hành động)

Mỗi ngày, Agent chỉ có thể đặt mua thêm một số lượng hàng nhất định:

```
action = order_quantity thuộc {0, 1, 2, 3, 4, 5}
```

- 0: Không đặt thêm hàng.
- 1..5: Đặt mua thêm từ 1 đến 5 đơn vị hàng.

C. Reward Function (Hàm phần thưởng)

Phần thưởng tức thời nhận được mỗi ngày được tính toán như sau:

```
Reward (r) = revenue - purchase_cost - holding_cost - stockout_penalty
```

Chi tiết các thành phần kinh tế:

1. Revenue (Doanh thu):

```
revenue = sold_units * SELL_PRICE  
sold_units = min(inventory_start_of_day, demand)  
SELL_PRICE = 10.0
```

2. Purchase Cost (Chi phí mua hàng):

```
purchase_cost = action * BUY_PRICE  
BUY_PRICE = 5.0
```

3. Holding Cost (Chi phí lưu kho):

Tính trên lượng hàng còn tồn lại trong kho cuối ngày sau khi bán:

```
holding_cost = inventory_end_of_day * HOLDING_COST  
inventory_end_of_day = inventory_start_of_day - sold_units  
HOLDING_COST = 0.5
```

4. Stockout Penalty (Chi phí phạt thiếu hàng):

Phạt khi nhu cầu của khách hàng lớn hơn lượng tồn kho thực tế:

```
stockout_penalty = unmet_demand * STOCKOUT_COST  
unmet_demand = max(0, demand - inventory_start_of_day)  
STOCKOUT_COST = 3.0
```

D. Transition Dynamics (Động lực học môi trường)

Chu trình chuyển đổi trạng thái của mỗi ngày kinh doanh diễn ra như sau:

1. **Đầu ngày:** Nhận hàng đang chờ giao từ ngày hôm trước:

```
inventory = min(inventory + pending_order, MAX_INVENTORY) # MAX_INVENTORY = 20
```

2. **Ra quyết định:** Agent chọn hành động **action** -> đơn hàng chờ giao ngày mai được thiết lập:

```
pending_order = action
```

3. **Phát sinh nhu cầu (Demand):** Nhu cầu thực tế được sinh ngẫu nhiên từ phân phối xác suất rời rạc dựa vào xu hướng **demand_regime**:

- **Low Regime:** Nhu cầu tập trung quanh mức 1–2 đơn vị/ngày.
- **Medium Regime:** Nhu cầu tập trung quanh mức 3–4 đơn vị/ngày.
- **High Regime:** Nhu cầu tập trung quanh mức 5–6 đơn vị/ngày.

4. **Bán lẻ:**

```
sold = min(inventory, demand)
inventory = inventory - sold
```

5. **Cuối ngày:** Tính toán Reward **r** nhận được.

6. **Chuyển thứ:** Cập nhật ngày tiếp theo trong tuần:

```
day_of_week = (day_of_week + 1) % 7
```

7. **Chuyển xu hướng thị trường:** Xu hướng nhu cầu **demand_regime** chuyển đổi ngẫu nhiên sang ngày hôm sau dựa trên xích Markov với xác suất tự giữ nguyên xu hướng rất cao (~85%–90%).

E. Terminal và Truncated (Điều kiện kết thúc)

- **Terminal (Kết thúc tự nhiên):** Mỗi episode kéo dài đúng 30 ngày (ngày thứ 30 kết thúc thì **terminated = True**).
- **Truncated (Cắt ngắn):** Không áp dụng (**truncated = False**).

F. Giả định Markov (Markov Assumption Validity)

Giả định Markov cho rằng trạng thái kế tiếp s_{t+1} và phần thưởng r_{t+1} chỉ phụ thuộc vào trạng thái hiện tại s_t và hành động hiện tại a_t . Giả định này **hoàn toàn hợp lệ** trong môi trường này vì:

- Tồn kho hiện tại đã bao gồm toàn bộ lịch sử bán hàng và nhập hàng trước đó.
- Hàng đang chờ giao (**pending_order**) lưu giữ thông tin của đơn hàng đặt ngày hôm trước (độ trễ logistics bằng đúng 1 ngày).
- Xu hướng thị trường (**demand_regime**) chuyển trạng thái chỉ phụ thuộc vào regime hiện tại (tính chất của xích Markov bậc 1).
- Thứ trong tuần (**day_of_week**) tăng tuần hoàn không phụ thuộc lịch sử xa hơn.

3. THUẬT TOÁN (ALGORITHMS IMPLEMENTATION)

Dự án tự cài đặt 5 tác nhân (Agents):

A. Random Agent (Baseline)

Lựa chọn lượng hàng đặt mua hoàn toàn ngẫu nhiên tại mỗi bước thời gian:

- Chọn ngẫu nhiên đồng đều **action** từ tập $\{0, 1, 2, 3, 4, 5\}$.

B. Heuristic Agent (Baseline)

1. **Always Order 2:** Mỗi ngày luôn đặt mua đúng 2 đơn vị hàng hóa bất kể trạng thái tồn kho ra sao.

2. **Reorder Threshold:**

- Nếu tồn kho hiện tại rơi xuống dưới 5 đơn vị: Quyết định đặt tối đa 5 đơn vị hàng.
- Ngược lại: Đặt 0 đơn vị hàng.

C. Q-Learning Agent (Off-policy TD Control)

Cập nhật giá trị hành động $Q(s, a)$ dựa trên hành động tốt nhất có thể ở trạng thái tiếp theo:

```
Q(s, a) <- Q(s, a) + alpha * [ TD_Target - Q(s, a) ]  
Trong đó:  
TD_Target = r + gamma * max_a'( Q(s', a') )   (nếu s' không phải terminal)  
TD_Target = r                                   (nếu s' là terminal)
```

- **alpha:** Tốc độ học (Learning rate) — Tỷ lệ chấp nhận thông tin mới.
- **gamma:** Hệ số chiết khấu (Discount factor) — Tầm quan trọng của lợi nhuận tương lai.
- **TD Error** (Sai số TD) = $TD_Target - Q(s, a)$.

D. SARSA Agent (On-policy TD Control)

Cập nhật $Q(s, a)$ dựa trên hành động thực tế tiếp theo **next_action** được chọn bởi chính sách khám phá hiện hành của tác nhân tại trạng thái tiếp theo **s'**:

```
Q(s, a) <- Q(s, a) + alpha * [ TD_Target - Q(s, a) ]  
Trong đó:  
TD_Target = r + gamma * Q(s', next_action)   (nếu s' không phải terminal)  
TD_Target = r                                   (nếu s' là terminal)
```

E. Double Q-Learning Agent (Off-policy, giảm maximization bias)

Sử dụng hai bảng Q độc lập (Q_1 và Q_2). Epsilon-greedy chọn hành động dựa trên tổng giá trị $Q_1(s, a) + Q_2(s, a)$.

Mỗi bước cập nhật, tung đồng xu chọn ngẫu nhiên bảng cập nhật (xác suất 50%):

- **Nếu cập nhật Q1:** Dùng Q1 để chọn hành động tốt nhất ở trạng thái tiếp theo, nhưng dùng bảng Q2 để ước lượng giá trị của hành động đó:

```
best_action = argmax_a Q1(s', a)
TD_Target = r + gamma * Q2(s', best_action)
Q1(s, a) <- Q1(s, a) + alpha * [ TD_Target - Q1(s, a) ]
```

- **Nếu cập nhật Q2:** Dùng Q2 để chọn hành động tốt nhất ở trạng thái tiếp theo, nhưng dùng bảng Q1 để ước lượng giá trị của hành động đó:

```
best_action = argmax_a Q2(s', a)
TD_Target = r + gamma * Q1(s', best_action)
Q2(s, a) <- Q2(s, a) + alpha * [ TD_Target - Q2(s, a) ]
```

F. Epsilon-greedy Exploration và Evaluation Mode

- **Chính sách chọn hành động:**
- Với xác suất **Epsilon**: Chọn ngẫu nhiên hành động từ $\{0..5\}$ để khám phá môi trường.
- Với xác suất $1 - \text{Epsilon}$: Chọn hành động tham lam tối ưu $\text{argmax}_a Q(s, a)$ để khai thác.
- **Epsilon Decay Schedule:** Epsilon giảm dần theo từng bước thời gian:

```
Epsilon = max(Epsilon_End, Epsilon * Epsilon_Decay)
Với: Epsilon_Start = 1.0, Epsilon_End = 0.05, Epsilon_Decay = 0.999993
```

- **Evaluation Mode (Chế độ đánh giá):** Khi đánh giá tác nhân, ta tắt hoàn toàn cơ chế ngẫu nhiên bằng cách set **Epsilon** = 0. Tác nhân chỉ thực hiện hành động tham lam tối ưu và không cập nhật trọng số bảng Q.

4. BỘ KIỂM THỬ MÔI TRƯỜNG (UNIT TESTS)

Dự án cài đặt bộ kiểm thử tự động toàn diện gồm 50 bài test chạy qua Pytest tại thư mục `tests/`:

A. Kiểm thử Biên (Test Boundary) — [test_env.py](file:///d:/HocTap/HocMayNangCao/tests/test_env.py)

- **test_zero_inventory_zero_pending:** Kiểm tra xem khi kho rỗng và không có đơn hàng nào đang về, môi trường có hoạt động bình thường không, chi phí phạt stockout có bị tính đúng khi phát sinh nhu cầu.
- **test_max_inventory_max_order:** Kiểm tra giới hạn chứa kho. Đảm bảo nếu lượng hàng nhận cộng tồn kho vượt quá 20 đơn vị, lượng tồn kho đầu ngày sẽ bị cắt biên (capped) ở mức tối đa là 20.

- **test_inventory_never_negative**: Đảm bảo lượng tồn kho không bao giờ bị âm dưới bất kỳ chuỗi nhu cầu nào.

B. Kiểm thử Hành động không hợp lệ (Test Invalid Action) — [test_env.py](file:///d:/HocTap/HocMayNangCao/tests/test_env.py)

- **test_invalid_action_raises**: Đảm bảo môi trường ném ra lỗi `AssertionError` khi Agent cố tình truyền hành động sai quy định như `-1` hoặc `6`.

C. Kiểm thử Trạng thái kết thúc (Test Terminal State) — [test_env.py](file:///d:/HocTap/HocMayNangCao/tests/test_env.py)

- **test_episode_terminates_at_30_days**: Đảm bảo môi trường trả về cờ `terminated = True` tại đúng bước thời gian thứ 30 và không cho phép chạy quá thời lượng này.

D. Kiểm thử Hạt giống ngẫu nhiên (Test Seed) — [test_env.py](file:///d:/HocTap/HocMayNangCao/tests/test_env.py)

- **test_same_seed_same_trajectory**: Xác minh hai môi trường độc lập được khởi tạo bằng cùng một `seed` sẽ tạo ra chuỗi nhu cầu và phần thưởng giống hệt nhau khi thực hiện cùng chuỗi hành động.

- **test_different_seeds_different_trajectories**: Xác minh các hạt giống ngẫu nhiên khác nhau sinh ra quỹ đạo khác nhau.

E. Kiểm thử Mã hóa/Giải mã (Test Encoder/Decoder) — [test_encoder.py](file:///d:/HocTap/HocMayNangCao/tests/test_encoder.py)

- **test_roundtrip_all_states**: Chạy vòng lặp kiểm tra toàn bộ 2,646 trạng thái, đảm bảo giải mã trạng thái sau khi mã hóa sẽ khớp hoàn toàn 100% với trạng thái tuple ban đầu.

- **test_encoder_unique**: Đảm bảo không xảy ra hiện tượng trùng mã (2 trạng thái tuple khác nhau có cùng 1 index).

F. Kiểm thử Hàm Phần thưởng (Test Rewards) — [test_rewards.py](file:///d:/HocTap/HocMayNangCao/tests/test_rewards.py)

- **test_reward_components_add_up**: Xác minh tổng các thành phần chi tiết (revenue, purchase, holding, stockout) khớp hoàn toàn với giá trị reward trả về từ hàm `step`.

- **test_ordering_too_much_reduces_profit**: Đảm bảo chính sách nhập hàng bữa bãi (luôn nhập 5 đơn vị hàng mỗi ngày) sẽ mang lại lợi nhuận thấp hơn do chi phí lưu kho tăng cao, chứng minh hàm reward phạt đúng hành vi dư thừa tồn kho.

5. THỰC NGHIỆM & PHÂN TÍCH (EVALUATION PIPELINE)

A. Quy trình thực nghiệm chuẩn hóa

- **Hạt giống ngẫu nhiên:** Mỗi thuật toán được huấn luyện độc lập trên **10 seeds khác nhau** (từ seed 0 đến seed 9).
- **Training Budget:** Mỗi seed được huấn luyện đồng đều trong vòng **20,000 episodes**.
- **Epsilon schedule:** Epsilon bắt đầu từ 1.0 giảm dần về 0.05 theo tỷ lệ decay 0.999993 per-step.
- **Chế độ đánh giá:** Sau khi huấn luyện, model tốt nhất của mỗi seed được đánh giá qua 100 episodes độc lập với Epsilon = 0.

B. Các chỉ số đo lường thống kê (Mean +- Std)

Kết quả đánh giá trên 10 seeds độc lập được tổng hợp thành bảng thống kê (dữ liệu mẫu từ `evaluation_results.json`):

Thuật toán / Thuật toán	Lợi nhuận	Tỉ lệ hết hàng (Std)	Tỉ lệ thành công (Std)	Tồn kho trung bình (Std)	Số bước (Avg)	Số ngày huấn luyện (Std)
Random Agent	-18.73 +- 50.84	0.5097 +- 0.0911	37.0%	2.54 +- 0.98	30.0 +- 0.0	15.29 +- 2.73
Always Order 2	200.75 +- 34.66	0.4497 +- 0.0886	100.0%	2.21 +- 0.72	30.0 +- 0.0	13.49 +- 2.66
Reorder Threshold	402.10 +- 27.67	0.0753 +- 0.0487	100.0%	7.91 +- 0.93	30.0 +- 0.0	2.26 +- 1.46
Q-Learning	321.41 +- 38.35	0.2012 +- 0.0745	100.0%	4.67 +- 0.81	30.0 +- 0.0	6.04 +- 2.23
SARSA	340.23 +- 35.12	0.1654 +- 0.0620	100.0%	5.12 +- 0.76	30.0 +- 0.0	4.96 +- 1.86
Double Q-Learning	328.60 +- 37.11	0.1876 +- 0.0698	100.0%	4.95 +- 0.79	30.0 +- 0.0	5.63 +- 2.09

g

C. Phân tích kết quả thực nghiệm

1. **Tính ổn định:** Độ lệch chuẩn (std) của các thuật toán RL ở mức chấp nhận được (~35.0 trên tổng Profit trung bình ~330), cho thấy thuật toán hội tụ tương đối ổn định qua các seed ngẫu nhiên khác nhau.

2. So sánh hiệu năng:

- Tất cả các RL Agent và Heuristic Agent đều vượt trội hoàn toàn so với **Random Agent**

(Lợi nhuận âm, tỉ lệ stockout lên tới 50%).

- **SARSA** đạt kết quả tốt nhất trong nhóm RL (Lợi nhuận ~340), theo sau là **Double Q-Learning** (~328) và **Q-Learning** (~321).

- **Heuristic Reorder Threshold** (Đặt hàng khi tồn kho dưới 5) đạt lợi nhuận rất cao (~402) do cấu trúc nhu cầu và chi phí lưu kho của đề tài khá tuyến tính. Tuy nhiên, ưu điểm của RL Agent là tính tự thích nghi mà không cần thiết lập ngưỡng cứng từ con người.

3. Khả năng tổng quát hóa (Weekend Surge):

Khi kiểm tra trên phân phối nhu cầu tăng đột biến vào cuối tuần (`weekend_surge = True`), các tác nhân học máy tăng cường (đặc biệt là Double Q-Learning) thể hiện khả năng thích ứng linh hoạt hơn, hạn chế được sự gia tăng đột biến của chi phí phạt hết hàng nhờ thông tin trạng thái `day_of_week` được tích hợp sẵn.

6. CHI TIẾT CẤU TRÚC MÃ NGUỒN DỰ ÁN

Mã nguồn được tổ chức sạch sẽ theo đúng chuẩn phân lớp của giảng viên:

```
HocMayNangCao/
├── envs/
│   ├── base_env.py          <- Lớp cơ sở định nghĩa giao diện chuẩn (reset, step, render, sta...
│   └── custom_env.py        <- Môi trường mô phỏng InventoryEnv chính thức, tính toán chuyển ...
├── agents/
│   ├── random_agent.py      <- Tác nhân đặt hàng ngẫu nhiên để làm baseline so sánh
│   ├── heuristic_agent.py   <- Các quy tắc heuristic cứng: AlwaysOrder2 và ReorderThreshold
│   ├── q_learning.py        <- Tabular Q-Learning off-policy với epsilon-greedy
│   ├── sarsa.py             <- Tabular SARSA on-policy cập nhật dựa trên hành động thực tế ti...
│   └── double_q_learning.py  <- Double Q-Learning sử dụng hai bảng Q độc lập chống lỗi maximiz...
├── experiments/
│   ├── configs.yaml         <- File cấu hình các siêu tham số huấn luyện (alpha, gamma, epsil...
│   ├── train.py             <- Kịch bản train đồng thời 3 agents RL x 10 seeds, xuất các file...
│   ├── evaluate.py          <- Kịch bản đánh giá 6 agents x 10 seeds, tính mean/std và kiểm t...
│   └── sweep.py             <- Script grid search tìm bộ tham số tối ưu (alpha, gamma, decay)...
├── tests/
│   ├── test_env.py          <- Unit test biên tồn kho, ngày kết thúc và tính tái lập của hạt ...
│   ├── test_encoder.py      <- Unit test cho hàm chuyển đổi qua lại 2,646 trạng thái
│   └── test_rewards.py      <- Unit test đảm bảo các chi phí phạt hoạt động đúng toán học
├── visualization/
│   ├── render.py            <- Tiện ích hiển thị terminal trạng thái theo ngày
│   └── plots.py             <- Script tạo ra 9 biểu đồ chất lượng cao (learning curves, so sá...
└── dashboard/
    └── app.py               <- Giao diện demo Web Dashboard xây dựng bằng Streamlit
```

```
| requirements.txt  
| README.md
```

```
<- Danh sách các thư viện phụ thuộc (numpy, matplotlib, pandas, s...  
<- Tài liệu hướng dẫn sử dụng và vận hành dự án
```

7. HƯỚNG DẪN VẬN HÀNH VÀ CHẠY DỰ ÁN (WINDOWS)

Đảm bảo bạn chạy các lệnh này tại thư mục gốc của dự án trong cửa sổ terminal.

Bước 1: Khởi tạo và cài đặt môi trường

Chạy lệnh đổi font hiển thị trên Windows Terminal để không bị lỗi ký tự Unicode và cài đặt các thư viện:

```
chcp 65001  
py -m pip install -r requirements.txt
```

Bước 2: Chạy kiểm thử tự động

Xác thực tính đúng đắn của toàn bộ logic hệ thống qua 50 bài kiểm tra:

```
py -m pytest tests/ -v
```

Bước 3: Huấn luyện các Agent RL

Chạy huấn luyện cho Q-learning, SARSA, Double Q-learning trên 10 seeds:

```
py experiments/train.py
```

(Quá trình này mất khoảng 10–15 phút. Kết quả các bảng Q-table đã học sẽ được ghi lại trong thư mục `results/`).

Bước 4: Chạy Đánh giá

So sánh tất cả các tác nhân dựa trên các file Q-table đã lưu:

```
py experiments/evaluate.py
```

Bước 5: Tạo các biểu đồ báo cáo

Tự động vẽ và lưu các biểu đồ so sánh, biểu đồ phân phối hành động đặt hàng vào thư mục `reports/figures/`:

```
py visualization/plots.py
```

Bước 6: Khởi chạy Web Dashboard

Mở giao diện web tương tác để thuyết trình demo trực quan trước giảng viên:

```
py -m streamlit run dashboard/app.py
```

(Giao diện web tự động mở tại địa chỉ local: `http://localhost:8501`).